

Bom dia/boa tarde, professor. Vou apresentar a minha dissertação de mestrado, intitulada *Explicando Decisões de LLMs via Otimização Inversa*, sob orientação do Prof. Raul Fonseca Neto. A apresentação está organizada em dois blocos: no primeiro, o LLM é a fonte de decisões e nós tentamos recuperar a métrica por trás delas; no segundo, o LLM passa a ser o aprendiz, recebendo rótulos de um perito externo.

LLMs são caixas-pretas: têm bilhões de parâmetros e respondem em texto livre. Quando se pede explicação em linguagem natural, o que se recebe é uma racionalização, não a causa real da decisão. Em aplicações críticas — saúde, crédito, direito — isso é insuficiente. A pergunta central deste trabalho é: o LLM tem um processo decisório estável e aprendível? A proposta é inverter a abordagem usual: em vez de pedir explicação, observamos as decisões do LLM e *recuperamos matematicamente* o critério por trás delas via otimização inversa.

Cinco hipóteses guiam o trabalho. **H1**: o LLM mantém um critério decisório estável; uma métrica W aprendida no Problema A deve prever as decisões em B e C. **H2**: exemplos few-shot amplificam essa concordância. **H3**: o LLM consegue aprender o critério de um perito externo via in-context learning. **H4**, originalmente: exemplos difíceis ensinam mais que fáceis — foi *refutada*. **H5**: os resultados são reproduzíveis entre sementes. Vou voltar a cada uma dessas hipóteses ao longo da apresentação.

Slide 4 — Abordagem (visão geral)

A abordagem em uma frase: trato cada classificação do LLM como saída de um classificador nearest-centroid ponderado por uma métrica de Mahalanobis *diagonal*. Cada feature recebe um peso w_j , e a distância ao centróide é a soma ponderada dos quadrados. O problema inverso é: dados os pontos e as classificações do LLM, encontrar W que reproduza o máximo dessas decisões. Uso *dois algoritmos independentes* — Perceptron Estruturado e NNLS — para garantir robustez. Se ambos chegam ao mesmo W , a evidência não depende do método de estimação.

Para a banca, formalizo: a distância é $d_W(x, c) = \sum w_j(x_j - c_j)^2$. O classificador é o nearest-centroid: argmin sobre as classes. Conexão com Mahalanobis cheia: W diagonal corresponde a Σ^{-1} quando as features são independentes — é a suposição naive Bayes embutida na métrica. A margem geométrica é a diferença de distância ao centróide errado menos o certo. A escolha da matriz diagonal é proposital: $O(d)$ parâmetros em vez de $O(d^2)$, identificável com poucos dados e diretamente interpretável — um peso por feature.

Antes de qualquer experimento, quero apresentar os seis problemas que aparecem no trabalho. Quatro deles são gaussianas anisotrópicas *lineares*: A serve de treino da métrica, B e C são testes de consistência com rotações opostas, D é o perito externo da Fase D. Dois são genuinamente *não-lineares*: a meia-lua sintética e a base real peso $\times$ altura, do orientador. Nos próximos seis slides, detalho cada um: propósito, como foi gerado passo a passo, parâmetros numéricos, o código e a justificativa de cada escolha.

O Problema A é a bancada de treinamento — onde o LLM classifica em zero-shot e onde aprendemos $\hat{W}$. Foi gerado com `sklearn.datasets.make_blobs`: dois centróides em $(-2, 0)$ e $(2, 0)$, distantes 4 unidades na horizontal, com desvio padrão 1,2 isotrópico, totalizando 150 amostras balanceadas. Escolhi essa configuração porque o problema fica linearmente separável em x_1 com fronteira ideal em $x_1 = 0$; o desvio padrão em torno da metade da distância entre centróides cria sobreposição parcial — não é trivial, mas é fácil para o LLM. O código é três linhas: chamada para `make_blobs` com esses parâmetros.

O Problema B testa se a métrica aprendida em A funciona em uma geometria *ligeiramente diferente*. Mesma função `make_blobs`, mas os centróides agora estão deslocados verticalmente: $(-2; +0,8)$ e $(2; -0,8)$. Isso é o que chamo de rotação horária — a classe da esquerda subiu, a da direita desceu. Mantenho o mesmo `cluster_std=1,2`; coloco apenas 100 amostras, já que B não serve para treinar W . A racional é: se o LLM tem critério estável, sua métrica deve acompanhar esse pequeno deslocamento sem grande perda. A discriminabilidade horizontal continua intacta; adicionamos só um componente vertical para testar transferência.

O Problema C foi alterado após a reunião de 30 de abril — o professor pediu que C não fosse uma simples cópia de B com mais distorção, mas uma rotação no *sentido inverso*. Então os centróides ficaram em $(-2; -1,5)$ e $(2; +1,5)$, anti-horário: agora a classe 0 desce e a classe 1 sobe. A magnitude da rotação é maior — $|\pm 1,5|$ contra $|\pm 0,8|$ em B — para testar generalização em uma transformação mais severa. E a direção oposta serve como controle: se houvesse viés sistemático do LLM ou da métrica, ele apareceria em apenas um dos dois problemas, não em ambos.

No Problema D os papéis se invertem. Aqui não é o LLM que rotula — é um perito externo, com métrica conhecida $W_{\text{expert}} = [0,3; 1,5]$, que rotula os 150 pontos. O LLM recebe alguns desses rótulos como exemplos few-shot e tenta reproduzir o critério do perito. Os centróides estão em $(-1,5; +1)$ e $(1,5; -1)$, com desvio 1,3 — pontos um pouco mais espalhados que em A. A escolha do W_{expert} é proposital: como w_2 é cinco vezes maior que w_1 , a fronteira fica quase horizontal — pouco intuitiva geometricamente. O LLM precisa capturar o padrão a partir dos exemplos; não basta usar Euclidiana.

A meia-lua foi adicionada depois da reunião de 30 de abril — o professor disse, e cito: *a base da meia-lua você acha em qualquer lugar aí*. É o caso canônico de fronteira não-linear da literatura de Machine Learning. Aqui não uso `make_blobs`; uso `make_moons` com 150 amostras e `noise=0,15`, que é o valor clássico — gera duas semi-luas reconhecíveis com sobreposição parcial nas extremidades. É um problema-controle positivo: esperamos que adicionar features quadráticas (R3 ou R4) melhore a acurácia, e a literatura confirma isso. Se nossa metodologia não detectar ganho aqui, algo está errado.

Este é o único problema com dados reais no trabalho. A base foi enviada pelo orientador no e-mail das 19:15 do dia 30/04, junto com a forma do classificador ótimo. São 100 amostras, 50 homens e 50 mulheres, com features já normalizadas no intervalo $[-1, 1]$ pelo formato do NeuroSolutions. O classificador ótimo bayesiano fornecido pelo orientador tem forma elíptica: $f(x) = x_2^2 - x_2 + x_1^2 - x_1 + \text{cte}$. Por isso a augmentação R4 com x_1^2 e x_2^2 é perfeita aqui. Só que com 100 amostras e 4 features, estamos no limite da rule-of-thumb 25:1, e isso vai gerar o *paradoxo do overfitting* no slide 36.

Sobre a infraestrutura: o modelo é gpt-4o-mini da OpenAI, temperatura zero, com três sementes [42, 123, 7] e três repetições por configuração — o que dá nove observações para bootstrap. Os tamanhos few-shot são 0, 5, 10, 20 e 40. Quatro estratégias de exemplos: easy, hard, mixed, random. Três experts na Fase D. Os hiperparâmetros do Perceptron seguem o artigo de origem, CILAMCE 2017: $\eta = 0,001$, $C = 1,0$ — escolhi $C = 1$ porque foi o valor mais conservador da faixa do artigo. Concorrência assíncrona com semáforo de 10 e timeout de 60 segundos. No total, esta execução fez 115 mil chamadas de LLM em cerca de três horas.

Slide 14 — Por que múltiplas seeds?

A justificativa de usar três sementes e três repetições é quádrupla. Primeiro, a geração de dados sintéticos é estocástica — uma seed pode produzir um arranjo atípico de pontos. Segundo, o LLM tem estocasticidade residual mesmo com temperatura zero, por causa do batching em GPU. Terceiro, múltiplas seeds permitem bootstrap 95% CI, teste de H5 e descarte de achados que dependem de uma seed específica. Quarto, é prática padrão em ML empírico para evitar p-hacking de seed. Com nove observações por configuração, temos margem para inferência estatística robusta.

O primeiro algoritmo é o Perceptron Estruturado com relaxação de margem, do artigo de Coelho, Borges e Fonseca Neto no CILAMCE 2017. A formulação é a Equação 28 do artigo: maximizar γ sujeito a que a diferença de distâncias mais $\lambda\alpha_i$ seja maior ou igual a $\gamma\|w\|$, com $w \geq 0$. A regra de correção (Equação 29) atualiza w por ponto violado. Há busca binária em γ : incrementamos γ em $\Delta = 0,05$ enquanto o algoritmo converge. Cito literalmente o artigo: *a taxa de aprendizado η foi 0,001 e a constante C variou de 1 até 0,1*. Quando não converge, faz fallback best-effort.

O segundo algoritmo é mais simples: mínimos quadrados não-negativos, formulado como $\min \|Aw - b\|^2$ sujeito a $w \geq 0$. A matriz A é construída com as diferenças de quadrados de coordenadas: linha i tem $(x_{ij} - c_{kj})^2 - (x_{ij} - c_{\ell j})^2$. O vetor b é um vetor de uns — a margem-alvo. Como a razão entre componentes de w é invariante à escala de b , esse valor não importa. O algoritmo é o método de conjunto ativo de Lawson-Hanson, disponível no `scipy.optimize.nnls`. A grande vantagem sobre o Perceptron: zero hiperparâmetros, determinístico, garantido a encontrar o ótimo global do problema convexo.

Vale destacar como tornamos a coleta robusta. O parser tem sete camadas em cascata para lidar com respostas malformadas do LLM: regex direto, busca por keyword, contexto, similaridade de string e assim por diante. Se ainda assim falha, tentamos até cinco vezes reenviando com reforço da instrução. Só no último recurso usamos um fallback determinístico baseado em hash MD5 das coordenadas — substituí a aritmética modular original porque ela introduzia viés geométrico. Para velocidade, usamos semáforo assíncrono de 10 chamadas simultâneas, reduzindo o tempo em uma ordem de magnitude. Cada requisição tem timeout de 60s e wrap externo de 90s para evitar travamento.

Antes de aplicar Perceptron e NNLS ao LLM, faço um sanity check fundamental: e se o W verdadeiro fosse conhecido? Gero dados sintéticos rotulados por um W oráculo — por exemplo $[0,1; 2,0]$ — e checo se os dois algoritmos recuperam esse W . Os resultados confirmam: cosseno acima de 0,95 nas três configurações anisotrópicas testadas (x2 dominante, x1 dominante, anisotropia forte). Se os algoritmos falhassem aqui — com oráculo perfeito — qualquer achado sobre o LLM ficaria comprometido. É o piso de sanidade da metodologia.

O segundo sanity check é a transferência. Aprende-se W em uma configuração — digamos x_2 dominante — e aplica-se em outra, como x_1 dominante ou anisotropia extrema. A consistência cai, como esperado. Esse achado importa: mostra que a métrica e a geometria são específicas a cada problema. Não existe um W universal independente dos centróides. Quando a geometria muda, W tem de ser reaprendido. Isso justifica metodologicamente por que recalculamos centróides em B e C a partir das decisões do LLM em cada problema.

Daqui em diante entramos no Bloco I — o LLM como fonte de decisões. O esquema é simples: o LLM rotula, nós aprendemos W a partir das decisões dele. As Fases A, B e C cobrem os Problemas A, B e C lineares. As hipóteses testadas neste bloco são H1 (consistência) e H2 (few-shot amplifica concordância).

A Fase A funciona assim, passo a passo. O LLM recebe os 150 pontos do Problema A em zero-shot, sem nenhum exemplo. Para cada ponto, responde A ou B com temperatura zero. Essas 150 respostas formam meu par (X, y_{LLM}) . Aqui um detalhe metodológico crucial: os centróides são recalculados *a partir das respostas do LLM*, não da classe verdadeira. Isso é proposital — ancoramos a métrica no que o LLM acha que são as classes, não na verdade do problema. Depois, Perceptron Estruturado e NNLS aprendem W independentemente, e reportamos a fidelidade: que fração das decisões do LLM a métrica consegue reproduzir.

Para a seed 42, o W aprendido é bem assimétrico: w_1 por volta de 0,95 e w_2 por volta de 0,05, tanto no Perceptron quanto no NNLS. Isso significa que o LLM, mesmo sem exemplos, está usando majoritariamente x_1 para classificar — o que faz sentido, já que o Problema A é linearmente separável em x_1 . A fidelidade fica em torno de 83%: a métrica reproduz mais de quatro em cada cinco decisões do LLM. O gráfico mostra os pontos coloridos pelo erro da métrica — são basicamente os pontos próximos da fronteira.

Quando decompos os erros por região do plano, o padrão é bem claro. Próximo à fronteira — onde $|x_1| < 0,5$ — estão concentrados cerca de 60% dos erros. Na margem média, 30%. Longe da fronteira, só 10%. E o LLM também erra mais perto da fronteira: cai de 96% de acerto longe para 75% perto. Os erros do LLM e da métrica se concentram nas mesmas regiões — evidência indireta de que a métrica está modelando o LLM, não só o problema. Se modelasse só o problema, os erros estariam descorrelacionados.

Quando olhamos o W aprendido entre as três sementes, a distribuição é qualitativamente estável. w_1 sempre muito maior que w_2 . As razões são próximas, e o cosseno entre W 's de sementes diferentes é maior que 0,97. Isso sustenta H5 — estabilidade. Diferenças quantitativas existem, mas a interpretação do critério decisório do LLM é a mesma em todas as sementes.

Esse painel consolida visualmente o que acabei de dizer: as três sementes produzem resultados qualitativamente idênticos para fidelidade, W e métricas de avaliação. Os boxplots ficam compactos, sem outliers extremos. É a evidência visual mais direta para H5 — a estabilidade entre sementes. Se um colega rodar de novo este experimento com outra semente qualquer, deveria chegar a esse mesmo padrão.

Aqui comparo lado a lado Perceptron e NNLS no mesmo conjunto de decisões do LLM. O cosseno entre os W 's estimados é tipicamente acima de 0,97. Eles concordam quase perfeitamente. Isso é importante porque mostra que a evidência não depende do método de estimação — se o LLM tivesse um padrão decisório específico que só um dos algoritmos detectasse, teríamos um problema metodológico. Como ambos chegam ao mesmo W , o achado é robusto.

Faço também uma análise de sensibilidade do Perceptron aos seus três hiperparâmetros: η , C e Δ_γ . Vario cada um em duas ordens de magnitude em torno do default. A fidelidade fica entre 80 e 85% — robusta. A razão w_1/w_2 varia de 18 a 28, então a interpretação qualitativa ($w_1 \gg w_2$) não muda. Conclusão: dentro de uma faixa razoável de hiperparâmetros, os resultados são estáveis. Não há cherry-picking.

Aqui chega o ponto que o orientador certamente vai questionar, e quero adiantar a resposta. Fidelidade e acurácia são coisas *diferentes*. Fidelidade é o quanto a métrica reproduz o LLM. Acurácia LLM vs real é o quanto o LLM está correto. Na seed 42, Problema A, zero-shot: fidelidade é 83%, mas a acurácia do LLM contra o ground truth é 28%. O LLM literalmente inverteu as classes! E mesmo assim, a métrica reproduz suas decisões invertidas com 83% de fidelidade. Isso mostra que estamos medindo CONSISTÊNCIA INTERNA do LLM — H1 — não correção. Mesmo errando a verdade, o LLM é consistente consigo mesmo.

Agora as Fases B e C. A pergunta é: a métrica $\hat{W}$ aprendida em A generaliza para outras geometrias? O LLM classifica os pontos de B (e depois C) em zero-shot. Aplico $\hat{W}$ vinda da Fase A nos pontos de B, com centróides recalculados localmente em B a partir das respostas do LLM. Comparo as predições da métrica com as decisões do LLM em B. Reporto três métricas: consistência simples (% de concordância), Cohen's Kappa (que corrige por concordância ao acaso) e F1.

Os resultados mostram consistência tipicamente entre 75 e 85% nas Fases B e C, com Kappa entre 0,5 e 0,7 — concordância moderada a substancial. O ganho da Fase A para B/C com few-shot é de 5 a 10 pontos percentuais, conforme H2 previa. Vale notar que a Fase A já tinha 17% de erro (fidelidade 83%); B e C herdaram essa ruído base mais o efeito da rotação geométrica. Mesmo assim, $\hat{W}$ transfere. H1 é sustentada.

Este é um detalhe metodológico que o professor pode levantar. Em B e C, posso usar centróides locais (recalculados em B a partir das respostas do LLM em B) ou centróides transferidos (reaproveitados do Problema A). Testei os dois. Na seed 42, Fase B: locais dão 71% de consistência, transferidos dão 77%. Os transferidos podem ganhar quando a geometria de B é próxima de A. Mas como padrão usei os *locais* — são mais fiéis à decisão real do LLM em cada problema. É a escolha mais conservadora e a que evita “trapacear” reaproveitando informação de A.

Em termos interpretativos: Kappa entre 0,5 e 0,7 indica concordância moderada-substancial corrigida por acaso — não é ouro, mas está muito acima do que se espera por sorte. A Fase A tem fidelidade 83%; B e C herdam esse ruído base. O ganho zero-shot → few-shot, da ordem de 5 a 10 pontos percentuais, alinha-se com H2. H1 é sustentada: a métrica transfere com perda moderada, não colapso. É o resultado esperado se o LLM realmente tem um critério estável.

Para dimensionar o experimento: 115 mil e quinhentas chamadas de LLM nesta execução, em cerca de três horas de coleta, custando entre 30 e 50 dólares de API. As configurações são 3 seeds vezes 3 reps vezes 5 n_shots vezes 4 estratégias vezes 3 experts, dando nove observações por configuração — suficiente para bootstrap CI. Fidelidade na Fase A fica entre 82 e 85%, consistência em B entre 75 e 82%, em C entre 70 e 80%. É experimento de escala adequada para uma tese de mestrado.

Agora R3 e R4. Na reunião do dia 30, o orientador disse algo como: em A/B/C lineares, ir de 84% para 75% não deve ajudar; mas na meia-lua e no peso $\times$ altura, espera-se que ajude. R3 adiciona $x_1 x_2$ (kernel quadrático mínimo, hipérbole). R4 adiciona x_1^2 e x_2^2 — corresponde a uma elipse alinhada, que é exatamente a forma do classificador ótimo de peso $\times$ altura. O LLM só vê (x_1, x_2) ; o aumento de features acontece depois, na augmentação de X antes de aprender W .

Em A/B/C lineares, os ganhos do NNLS ao passar de R3 para R4 são baixos: cerca de +2,7% em A, +1,1% em B, +0,8% em C — praticamente zero, conforme o orientador previu. O Perceptron, porém, mostra +18,7%, o que é suspeito. Provavelmente é overfitting com 4 pesos para 150 amostras ou não-convergência da busca binária em γ com mais dimensões. Por isso uso o NNLS como evidência robusta nos problemas lineares. A conclusão: R3/R4 não ajuda em A/B/C, como o esperado. Isso prepara a transição para o Bloco II, onde testamos em problemas não-lineares de verdade.

Este é um achado contraintuitivo que vale apresentar. Em peso $\times$ altura, ao passar de 2 para 4 features, a fidelidade do Perceptron sobe quase 23 pontos percentuais — a métrica decora melhor o LLM. Mas a acurácia contra o rótulo real *cai* de 1 a 23 p.p. Como isso é possível? Porque o LLM em si erra muito com prompt neutro (x_1, x_2) , e a métrica com 4 pesos para 70 amostras de treino tem graus de liberdade suficientes para *decorar o ruído das decisões erradas do LLM*. É overfitting clássico. Mensagem para o orientador: subir n_{feat} só vale se houver não-linearidade real e dados suficientes.

Aqui testei se a ordem em que apresento as opções ao LLM — “A ou B” versus “B ou A”, “0 ou 1” versus “1 ou 0” — altera o resultado. A diferença observada é marginal: menos de 3 pontos percentuais em qualquer combinação. Esse viés existe, mas não é dominante. Não explica nenhum dos achados principais. Vale registrá-lo para descartar como confound.

Quatro variantes de prompt testadas. *Default*: o baseline atual. *Geometric*: contexto espacial explícito, falando em “grupos de pontos em um plano cartesiano”. *CoT*, chain-of-thought: pedindo “pense passo a passo antes de responder”. *Tabular*: coordenadas apresentadas como tabela markdown. Todas as quatro rodam as Fases A, B e C completas, em todas as seeds. Se W aprendido muda muito entre variantes, significa que o prompt confunde a medição e o critério não é tão estável quanto parecia.

O resultado é bom para a tese: o scatter de W entre as quatro variantes mostra que os pontos colapsam próximos uns dos outros. Variantes produzem W 's qualitativamente equivalentes; a razão w_1/w_2 varia menos de 5%. Isso é evidência adicional de H1: o critério está no LLM, não na forma do prompt. Trocar a redação mantém a mesma métrica.

Aqui muda o nome das features no prompt: (x_1, x_2) neutro, (altura, peso) semântico, (feature_1, feature_2) técnico. Em problemas sintéticos sem prior cultural, W é estável entre as variantes. Mas em problemas onde existe prior — por exemplo peso/altura, que está correlacionado com gênero em todo o corpus de treinamento do LLM — o W pode mover sensivelmente. Isso vai aparecer com toda força na aplicação da base real, na Parte 11. Posicionei este slide junto com as variantes de prompt porque conceitualmente é uma variante: mudar o nome das features é uma reformulação do prompt.

Mostro aqui o texto exato que chega ao LLM no zero-shot e no few-shot default. Isso é importante porque, para a banca avaliar o experimento, é essencial ver o que *de fato* chega ao modelo, não só uma descrição em alto nível. O zero-shot tem três linhas: contexto, ponto a classificar, formato da resposta. O few-shot adiciona cinco exemplos rotulados antes da pergunta. Trechos tomados do `llm_interactions.json` desta execução.

Aqui estão as três variantes lado a lado. O *geometric* prefixa a pergunta com a metáfora espacial: “você vê dois grupos de pontos”. O *CoT* insere instrução explícita de raciocinar passo a passo, com três passos enumerados. O *tabular* formata os exemplos como tabela markdown, simulando uma planilha. Todos os três terminam com a mesma instrução: “responda apenas A ou B”. A intenção é manter o sinal de saída controlado e variar só a forma de entrada.

Entramos agora no Bloco II. A inversão é completa: aqui um perito (ou métrica oráculo) rotula, e o LLM tenta reproduzir o critério dele a partir de exemplos few-shot. Cobre a Fase D nos Problemas D, peso $\times$ altura e meia-lua. As hipóteses primárias deste bloco são H3 (LLM como aprendiz), H4 (exemplos difíceis — que será refutada) e H5 (estabilidade). Os problemas D, E e peso $\times$ altura já foram apresentados nos slides 10, 11 e 12; aqui eu uso os experimentos sobre eles.

A Fase D passo a passo: o perito externo com $W_{\text{expert}} = [0,3; 1,5]$ rotula os 150 pontos do Problema D. Esse W produz uma fronteira quase horizontal, já que w_2 é cinco vezes maior que w_1 . Divido em treino e teste. Para cada n_{shot} de 0 a 40, seleciono exemplos do treino segundo uma das quatro estratégias e envio ao LLM como few-shot. Depois peço ao LLM para classificar cada ponto do teste. Reporto acurácia, Kappa e F1 do LLM contra os rótulos do perito.

As quatro estratégias de seleção são: *easy* — pontos com alta margem, longe da fronteira do perito; *hard* — pontos com baixa margem, próximos da fronteira; *mixed* — 50% *easy* e 50% *hard*; *random* — baseline. O gráfico mostra a distribuição espacial de cada estratégia. É importante visualizar isso porque a intuição de H4 era: *hard* tem mais informação, então devia ganhar. Vamos ver no próximo slide que não é bem assim.

As curvas de aprendizado mostram o LLM melhorando com n_{shot} em todas as estratégias — isso sustenta H3 fortemente. O ganho é particularmente forte entre 0 e 5 exemplos; depois desacelera. É o padrão clássico de in-context learning. O LLM está de fato aprendendo o critério do perito a partir dos exemplos, não só memorizando rótulos.

Aqui vem a refutação da H4. As curvas mostram que *easy* domina em n_{shot} pequeno, ao contrário do que a intuição a priori sugeria. *Hard* fica abaixo de *easy*, *mixed* e *random* em 5 e 10 exemplos. H4 é refutada — estatisticamente, como vou mostrar dois slides adiante. A intuição do active learning, importada de modelos treinados com gradiente, não se aplica diretamente a LLMs in-context.

Slide 48 — Por que easy vence hard

Por que isso acontece? Em modelos treinados com gradiente, exemplos ambíguos próximos da fronteira são informativos porque contêm sinal sobre onde colocar a fronteira. Mas o LLM in-context não tem mecanismo de gradiente; ele usa os exemplos como *âncoras de classe*. Pontos prototípicos (easy, alta margem) são âncoras claras: “é isto que A parece, e é isto que B parece”. Pontos ambíguos (hard) são confusos: “este aqui é A? quase poderia ser B...”. Isso confunde, não ajuda. É coerente com Lyu et al. 2023 sobre prompts canônicos.

A refutação é estatisticamente significativa. Em 5-shot, a diferença easy menos hard é +0,233 com IC95% de [+0,119, +0,336] — claramente não inclui zero. Em 10-shot, +0,201 com IC95% [+0,107, +0,293]. Mas em 20-shot, +0,072 com IC95% [-0,126, +0,210], que inclui zero — não significativo. Metodologia: bootstrap 10 mil reamostras e Wilcoxon signed-rank pareado. Insight extra: o efeito *desaparece* com mais exemplos. Então a refutação de H4 é específica do regime de poucos exemplos. Com 20 exemplos, easy e hard convergem.

Testei três experts diferentes na Fase D. `aniso_x2` com $W = [0,3,1,5]$ — fronteira horizontal. `aniso_x1` com $W = [1,5,0,3]$ — fronteira vertical. `euclidean` com $W = [1,1]$. O LLM aprende todos eles, mas se sai melhor no Euclidean — 90% em 5-shot, 95% em 40-shot. Faz sentido: é o critério mais natural. Os dois anisotrópicos são mais difíceis e ficam em torno de 78–80% em 5-shot, 87–89% em 40-shot. Isso sustenta H3 amplamente: o LLM tem flexibilidade para adaptar-se a diferentes critérios.

O experimento de diluição foi desenhado para testar uma hipótese específica: será que adicionar muitos easy “dilui” a informação dos hard fixos? Fixei 4 exemplos hard e fui adicionando easy progressivamente: 0, 2, 4, 10, 16, 20. A curva resultante é monotônica crescente — adicionar easy ajuda sempre, não há diluição. Isso reforça o achado da H4: hard simplesmente não são informativos para LLM in-context.

Aqui testo recency bias — a ordem dos exemplos few-shot importa? Usei os mesmos exemplos com quatro ordenações: classe 0 primeiro, classe 1 primeiro, embaralhado, alternando. Testado em n_{shot} de 5 a 40. A variância entre ordenações é menor que 4 pontos percentuais. O recency bias existe, mas é pequeno para o tamanho de prompt que usamos. Não explica resultados principais.

Antes de migrar para problemas não-lineares, comparo o LLM com classificadores clássicos. k-NN com k ajustado ao n_{shot} (máximo 5, sempre ímpar), Logistic Regression linear L2-regularizada, e SVM com kernel RBF. Protocolo crítico: os baselines são treinados nos *mesmos* exemplos few-shot dados ao LLM e avaliados no *mesmo* conjunto de teste. A pergunta é: o LLM faz algo que um classificador trivial não faria?

Padrões que emergem: em n_{shot} baixo (5–10), o LLM compete ou supera os baselines. Em n_{shot} alto (20–40), SVM e LR alcançam ou superam o LLM. Por estratégia: easy e random favorecem o LLM. Em hard, o LLM é o pior dos quatro. A mensagem para o orientador: o LLM não é universalmente superior. Sua vantagem aparece em regime de poucos dados e estratégias prototípicas. Com 40 exemplos, baselines clássicos podem ser melhores.

Na base real, faço duas variantes de prompt para o LLM. *Neutra*: as features são x_1 e x_2 — LLM não sabe que é peso e altura. *Semântica*: as features são *peso* e *altura* — LLM ativa o prior semântico do treinamento massivo. A pergunta é a mesma em ambas: *homem ou mulher*. Isso replica em base real o experimento de “nomes semânticos” do slide 40, e testa se o LLM realmente usa priors. Spoiler: usa, e bastante.

Aprendo W em três configurações de features: R2 com (x_1, x_2) , R3 com (x_1, x_2, x_1x_2) que representa uma hipérbole, e R4 com (x_1, x_2, x_1^2, x_2^2) que representa a elipse alinhada — exatamente a forma do classificador ótimo dado pelo orientador. O gráfico tem quatro painéis: fidelidade pelo Perceptron, fidelidade pelo NNLS, acurácia contra o real pelo Perceptron, acurácia contra o real pelo NNLS. O esperado seria ganho ao passar de R2 para R4, já que o problema é não-linear.

O e-mail das 22:04 do orientador pediu explicitamente: “apresentar também a acurácia da métrica para a rotulação da base de dados original”. Atendo a esse pedido nesta tabela. Em R2, fidelidade 70% e acurácia 60% — gap de 10 p.p. Em R3, fidelidade sobe para 75% mas acurácia cai para 55% — gap 20 p.p. Em R4, fidelidade 78% e acurácia 37% — gap explodindo para 41 p.p. É o sinal do paradoxo do overfitting do slide 36, agora na própria base real do professor. Subir features só ajuda se houver dados.

Identificada a melhor configuração de features, uso essa métrica como rotuladora para a Fase D em peso $\times$ altura. Os exemplos do prompt incluem o mesmo número de features. Reporto acurácia, Kappa e F1 do LLM ao longo de n_{shot} de 0 a 40. O gráfico tem dois painéis: LLM versus métrica e LLM versus real, ambos agrupados por variante de prompt — x_1/x_2 versus peso/altura. Mostra como o LLM se sai como aprendiz num problema do mundo real, não só sintético.

Aqui atendo a outro pedido específico da reunião, em torno de 35 minutos da diarização: comparar o LLM in-context com um Perceptron Estruturado treinado nos *mesmos* exemplos do expert. Para cada n_{shot} , treino um Perceptron e reporto sua acurácia em paralelo com a do LLM. O achado preliminar: em peso $\times$ altura com 5-shot, o Perceptron supera o LLM em cerca de 7 pontos percentuais (90% versus 83%). Quando há dados rotulados disponíveis, métrica clássica pode bater LLM in-context. O LLM ganha mesmo é onde não há dados.

Análise consolidada do showdown LLM vs Perceptron. Em peso $\times$ altura, o LLM ganha em todos os n_{shot} , de +2 a +11 p.p. Mas em x_1/x_2 neutro, o Perceptron ganha em n_{shot} 10 e 20 por 3–7 p.p. Na meia-lua, o LLM ganha de novo em n_{shot} 10, 20 e 40. Interpretação: o LLM ganha quando tem *priors semânticos* (peso/altura) E n_{shot} é pequeno. O Perceptron ganha quando as features são *neutras* (x_1/x_2) E n_{shot} é médio. Mensagem: a vantagem do LLM vem dos priors do treinamento massivo, não do mecanismo de in-context learning per se.

Mesma metodologia do slide 56, agora na meia-lua. Os achados na seed 42 com prompt x_1/x_2 : em R2, fidelidade Perceptron 71% e acurácia real 49%; em R3, fidelidade 69% mas acurácia salta para 57% — ganho de 8,6 p.p.; em R4, fidelidade 75% e acurácia 58%, ganho de 9,5 p.p. acumulado. *Aqui sim* aumentar features ajuda — a meia-lua é genuinamente não-linear, e há 150 amostras de treino suficientes para 3 ou 4 pesos sem overfit.

Mesma curva de aprendizado da Fase D externa, agora na meia-lua. O ganho de zero-shot para 5-shot é particularmente forte aqui — por volta de 20 pontos percentuais de acurácia. Faz sentido: a fronteira não-linear da meia-lua é difícil sem nenhum exemplo, mas com poucos exemplos o LLM “pega o jeito” rapidamente. H3 é sustentada com muita força neste problema.

Consolidando os ganhos $R2 \rightarrow R4$ nos três cenários: em A, B e C lineares, ganho zero p.p. — conforme o esperado. Na meia-lua sintética, +9,5 p.p. — conforme esperado. Em peso $\times$ altura real, ganho *negativo* de -1 a -23 p.p. — contrário ao esperado, mas explicado pelo overfitting com 100 amostras. O padrão: R4 ajuda em não-linearidade real *quando há dados suficientes*.

Três conclusões fechadas do Bloco II. Um: H3 sustentada com forte ganho zero-to-5-shot — 20 p.p. na meia-lua, 6,7 p.p. em peso/altura. Dois: H4 refutada com significância em n_{shot} 5 e 10, mas o efeito desaparece em 20. Três: R4 só ajuda em problemas *genuinamente não-lineares* E com dados suficientes — meia-lua sintética diz sim, peso/altura com 100 amostras diz não por overfitting, e A/B/C lineares dizem não como esperado.

O gráfico de robustez entre as três seeds mostra que a fidelidade na Fase A varia 83 ± 1 ponto percentual; consistência na Fase B, 78 ± 2 ; na Fase C, 74 ± 3 . Os desvios padrões são pequenos. O comportamento qualitativo é idêntico em todas as sementes. H5 é sustentada com folga.

Uso três testes nas métricas principais. Bootstrap 95% CI com 10 mil reamostras. Wilcoxon signed-rank pareado. Cohen's d para tamanho do efeito. Resultados de destaque: few-shot vs zero-shot na Fase D tem $\Delta = +0,18$ com IC95% $[+0,12, +0,24]$ e Cohen $d = 1,2$ (efeito grande). Easy vs hard em 5-shot: $\Delta = +0,23$, IC $[+0,12, +0,34]$, Cohen $d = 1,5$ — efeito grande. Cosseno entre W Perceptron e NNLS: 0,98. Wilcoxon $p < 0,001$ em todos os principais.

Três testes não-paramétricos adicionais. Point-biserial correlation entre confiança do LLM e acerto: $r = 0,42$, $p < 0,001$ — o LLM “sabe” quando tem certeza. Mann-Whitney U comparando margem em acertos versus erros: $U = 8420$, $p < 0,001$ — erros se concentram em pontos de baixa margem. Kruskal-Wallis testando se a ordem dos exemplos few-shot altera a distribuição de performance: $H = 2,1$, $p = 0,55$ — não altera significativamente, reforçando o slide 52.

Tenho dashboards exploratórios por seed — gráficos 16 a 21 no arquivo de execução — com visão granular de tudo: scatter dos problemas, mapa de acertos e erros, matrizes de confusão, análise de margem, W por algoritmo, dashboard consolidado. Três seeds, tudo em PNG, disponível se a banca quiser detalhe. Mostro aqui apenas as matrizes de confusão da seed 42 como exemplo — as outras são qualitativamente idênticas.

Atendo aqui ao adendo do e-mail das 22:06: scatter ponto-a-ponto das rotulações do LLM em cada configuração — por problema, variante de prompt e n_{shot} . Mostro como exemplo lado a lado peso $\times$ altura com prompt semântico (esquerda) e com prompt neutro x_1/x_2 (direita), ambos em 5-shot da seed 42. Os mesmos pontos são classificados de forma muito diferente. É a evidência visual mais direta do efeito do prior semântico.

Síntese final por bloco. Bloco I: H1 sustentada (Kappa entre 0,5 e 0,7); H2 sustentada (ganho 5–10 p.p. com few-shot). Bloco II: H3 sustentada (+20 p.p. na meia-lua de 0 para 5 exemplos); H4 *refutada* com significância ($p < 0,001$, Cohen $d = 1,5$) — exemplos fáceis vencem difíceis em n_{shot} pequeno. Transversal: H5 sustentada (desvio entre sementes menor que 3 p.p.). Quatro das cinco hipóteses sustentadas; H4 refutada com diagnóstico claro.

Cinco conclusões para fechar. Uma: existe um critério decisório estável no LLM, recuperável via otimização inversa — H1. Duas: few-shot amplifica concordância LLM-métrica — H2. Três: o LLM aprende critério de perito via in-context learning — H3 — mas baselines clássicos são competitivos com mais dados. Quatro: exemplos fáceis superam difíceis em few-shot pequeno; efeito desaparece com 20+ exemplos. Cinco: kernel quadrático só ajuda em problemas não-lineares E com dados suficientes; overfitting é real com 100 amostras e 4 pesos.

Limitações: a métrica diagonal não captura interações lineares entre features — W cheia ou kernel resolveria. Estudei só 2D; problemas reais são multidimensionais. Em peso $\times$ altura tenho apenas 100 amostras — pouco para 4 features. Um único LLM testado, embora Claude e Gemini estejam integrados no código. E não há controle de versão do LLM entre seeds — o provider pode mudar silenciosamente. Direções futuras: Mahalanobis cheia, kernel não-diagonal, bases UCI maiores, múltiplos LLMs em paralelo, regularização L1 em W para seleção automática de features.

Encerro com agradecimentos — ao orientador Prof. Raul Fonseca Neto, ao PPGCC da UFJF, à minha família. Estou à disposição da banca para perguntas. Tenho ainda dez slides de apêndice técnico cobrindo formulação matemática completa do Perceptron, equivalência com NNLS, análise de overfitting, conexões com a literatura e reprodutibilidade, caso surjam perguntas específicas.

Se o professor pedir detalhe matemático do Perceptron, este é o slide. A formulação primal é a Equação 28 do artigo: $\max \gamma$ sujeito a que $d_W(x_i, c_k) - d_W(x_i, c_\ell) + \lambda \alpha_i \geq \gamma \|w\|$, com $w \geq 0$ e $\alpha \geq 0$, onde $\lambda = 1/C$. A regra de correção por ponto violado é a Equação 29 — atualiza w , α e α_i . A atualização da margem entre níveis (Eq. 31): $\gamma^{(t+1)} = \gamma^{(t)} + \Delta$, ou multiplicação por 2. Hiperparâmetros da Seção 6 do artigo: $\eta = 0,001$, $C \in [0,1; 1,0]$. Conexão com Rosenblatt: $\gamma = 0$ e $\lambda = 0$ recuperam o perceptron clássico.

Sobre o NNLS: $\min \|Aw - b\|^2$ com $w \geq 0$. A linha i de A tem $(x_{ij} - c_{kj})^2 - (x_{ij} - c_{\ell j})^2$. b é vetor de uns — `target_margin = 1`. A razão w_1/w_2 é invariante à escala de b . Algoritmo: método de conjunto ativo de Lawson-Hanson de 1974. Solução única em problemas bem-condicionados (rank de A igual a d). Vantagens: zero hiperparâmetros, determinístico, garante ótimo global do problema convexo.

Resumo das diferenças: Perceptron usa gradiente projetado, tem três hiperparâmetros, não é determinístico (depende de inicialização) e não garante otimalidade global. NNLS usa programação quadrática via Lawson-Hanson, sem hiperparâmetros, determinístico, garante ótimo global. Mas na prática, os dois algoritmos são equivalentes: cosseno entre W 's acima de 0,97 nas nossas execuções. Razão w_1/w_2 diverge menos de 5%. É uma evidência empírica forte de consistência.

Caso emblemático de overfitting: peso $\times$ altura com prompt neutro (x_1, x_2) e 4 features. Fidelidade 78% — a métrica decora 78% das decisões do LLM. Mas acurácia contra rótulo real é 37% — o LLM erra 63% dos casos, pior que chute aleatório! Diagnóstico: a métrica com 4 pesos e 70 amostras de treino tem graus de liberdade suficientes para decorar o ruído das decisões erradas. Mitigações possíveis: regularização L1/L2, mais dados (bases UCI), validação cruzada, seleção automática de features.

Trade-off bias-variance na escolha de n_{feat} . Com 2 features: alto bias se o problema for não-linear, mas baixa variância. Com 4 features: baixo bias (capta ellipse), mas alta variância em $n = 100$. Rule of thumb “10 amostras por parâmetro” sugere que com 70 amostras de treino, 4 pesos está no limite. Recomendação para trabalhos futuros: curva de aprendizado contra n_{feat} para escolher inflexão, e regularização adaptativa.

Conexões com literatura de XAI: Ahuja e Orlin 2001 com a formulação clássica de otimização inversa em programação linear; Schultz e Joachims 2003 com margem SVM-style para metric learning; e Coelho, Borges, Fonseca Neto 2017 com o perceptron estruturado com relaxação — nosso algoritmo. Diferenciação: somos os primeiros a aplicar a LLMs com interface em texto, não numérica. Método black-box: não requer pesos do modelo. Comparado com SHAP, LIME ou attention visualization, nosso método é *independente de arquitetura*.

Referências-chave em in-context learning: Brown et al. 2020 introduziu GPT-3 e o paradigma few-shot. Min et al. 2022 mostrou que “ground truth labels don’t matter” e que o label space importa mais que a corretude individual. Wei et al. 2022 estudou habilidades emergentes em escala. Lyu et al. 2023 mostrou que prompts canônicos são mais efetivos. Nosso achado de refutação de H4 (easy vence hard) alinha com Lyu et al.: pontos prototípicos são melhores âncoras que pontos ambíguos.

Referências-chave em metric learning Mahalanobis: Xing et al. 2002 é o trabalho original. Davis et al. 2007 propôs ITML (Information-Theoretic Metric Learning). Schultz e Joachims 2003 trouxe o SVM-style margin. Para a justificativa da matriz diagonal especificamente, cito Liu et al. 2010 e Kedem et al. 2012 — são trabalhos que mostraram que matriz diagonal frequentemente compete com a cheia em alta dimensão.

Sobre reprodução: um único comando, `python src/dissertacao_mestrado.py`. Tudo é controlado por cerca de 20 constantes no topo do arquivo: `RANDOM_SEEDS`, `N_REPETICOES`, `FEW_SHOT_SIZES`, `MAX_CONCURRENCY`, flags como `RUN_HOMEM_MULHER`, `RUN_PROBLEM_E`. Tempo total: cerca de 3 horas coletando do LLM, mais 5 minutos de processamento estatístico. Custo de API: 30 a 50 dólares. Outputs: mais de 30 PNGs, 11 CSVs e um JSON com todas as 115 mil interações individuais com o LLM.

Por fim, limitações honestas de reprodutibilidade. O LLM evolui silenciosamente: a mesma versão “gpt-4o-mini” pode mudar entre execuções, conforme política do provider. Temperatura zero *não* garante determinismo — batching em GPU produz diferenças residuais em floats. Por isso usamos 3 seeds vezes 3 reps, totalizando 9 observações por configuração. A estocasticidade residual medida é tipicamente menor que 2% de variação em consistência entre reps da mesma seed. É o que é factível hoje em pesquisa com LLMs comerciais.